JAVA PROGRAMMING / DSA

Chapter 35: Sliding Window

Personal Notes

Sliding window is one of the highest-value patterns in coding interviews — small to learn, huge in payoff. It solves an entire family of problems about SUBARRAYS or SUBSTRINGS in O(n) time that would otherwise be $O(n^2)$ or worse with nested loops.

The core idea is simple: instead of recomputing the answer for every possible subarray from scratch, MAINTAIN a window that slides across the data. When you extend or shrink the window, update the answer INCREMENTALLY — reusing what you already know. That's it. The rest is pattern recognition.

1. What is a Sliding Window?

A sliding window is a range [left, right] on an array or string that we EXPAND and SHRINK to explore contiguous subsequences efficiently. Instead of examining every possible subarray separately, we walk one window across the whole input, adjusting as we go.

```
Array:  [1, 3, 2, 6, -1, 4, 1, 8, 2]
           _________
           window (left=0, right=2, size 3)

After sliding right by 1:
        [1, 3, 2, 6, -1, 4, 1, 8, 2]
              _________
              window (left=1, right=3)

Instead of RESCANNING every window, we update:
  - Remove the element that just LEFT the window
  - Add the element that just ENTERED the window
```

Why it's fast: Naive brute force considers every subarray from scratch — that's $O(n^2)$ or more. Sliding window's incremental update makes each step O(1), so the entire traversal is O(n). Same problem, dramatically better complexity.

2. Two Types of Sliding Window

Every sliding window problem falls into one of two categories:

2.1 Fixed-Size Window

The window size K is given. You slide it one step at a time across the array.

- "Find max sum of a subarray of size K"

- "Find average of every subarray of size K"
- "Count distinct elements in every window of size K"

2.2 Variable-Size Window

The window GROWS AND SHRINKS based on a condition. You expand while the condition holds, shrink when it breaks.

- "Longest substring with at most K distinct characters"
- "Smallest subarray with sum ≥ target"
- "Longest substring without repeating characters"

Quick Comparison

Aspect	Fixed-Size	Variable-Size
Window size	Fixed (K given)	Grows and shrinks dynamically
Trigger	Move both left and right together	Expand right while valid; shrink left when invalid
Loop shape	One clean for-loop over right	Outer while / for on right, inner while on left
Best for	"Every window of size K"	"Longest/shortest satisfying condition"

3. Fixed-Size Window — The Template

```
int left = 0;
int windowSum = 0;
int best = ...;   // some initial value

for (int right = 0; right < n; right++) {
    windowSum += arr[right];   // add the new element

    // Once the window is size K, start sliding
    if (right - left + 1 == k) {
        best = Math.max(best, windowSum);       // record result
        windowSum -= arr[left];                  // remove the oldest
        left++;                                   // slide left forward
    }
}
```

Notice the pattern: EXPAND (add arr[right]), CHECK size, RECORD result, SHRINK (remove arr[left]).

4. Fixed-Size — Problem 1: Max Sum Subarray of Size K

Given an array and integer K, find the maximum sum of any K consecutive elements.

Example

```
arr = [2, 1, 5, 1, 3, 2],  k = 3

Window at 0-2:  [2, 1, 5]     sum = 8
Window at 1-3:  [1, 5, 1]     sum = 7
Window at 2-4:  [5, 1, 3]     sum = 9    ← max
Window at 3-5:  [1, 3, 2]     sum = 6

Answer: 9
```

Code

```
public static int maxSum(int[] arr, int k) {
    int windowSum = 0, maxSum = 0;

    // Fill the first window
    for (int i = 0; i < k; i++) windowSum += arr[i];
    maxSum = windowSum;

    // Slide the window
    for (int right = k; right < arr.length; right++) {
        windowSum += arr[right] - arr[right - k];
        maxSum = Math.max(maxSum, windowSum);
    }

    return maxSum;
}

// Time: O(n) — every element added once, removed once
// Space: O(1)
```

> **The magic move:** windowSum += arr[right] - arr[right - k] does BOTH the add-new and remove-old in one line. This kind of "delta update" is what makes sliding window O(n) instead of O(n × k).

5. Fixed-Size — Problem 2: First Negative in Every Window of K

For each window of size K in the array, find the first negative number (or 0 if there's none).

```
import java.util.*;

public static List<Integer> firstNegative(int[] arr, int k) {
    List<Integer> result = new ArrayList<>();
    Deque<Integer> negatives = new ArrayDeque<>();   // indices of
negative numbers
```

```
    for (int right = 0; right < arr.length; right++) {
        if (arr[right] < 0) negatives.offer(right);

        // Once the window is full
        if (right >= k - 1) {
            int left = right - k + 1;

            // Remove indices that are no longer in the window
            while (!negatives.isEmpty() && negatives.peek() < left)
negatives.poll();

            result.add(negatives.isEmpty() ? 0 : arr[negatives.peek()]);
        }
    }

    return result;
}
```

A Deque (double-ended queue) is a common companion to sliding window when you need to track candidates ordered by position.

6. Variable-Size Window — The Template

Variable-size windows share the same skeleton — always. Once you memorize it, most problems become fill-in-the-blank.

```
int left = 0;
int best = ...;
// ... state you're tracking (sum, distinct count, freq map, etc.)

for (int right = 0; right < n; right++) {
    // 1. ADD arr[right] to the window state
    add(arr[right]);

    // 2. WHILE the window is INVALID, shrink from the left
    while (isInvalid()) {
        remove(arr[left]);
        left++;
    }

    // 3. RECORD the answer for this valid window
    best = Math.max(best, right - left + 1);
}
```

1. EXPAND — add arr[right] to the window
2. SHRINK — while the window is invalid, remove arr[left] and move left forward
3. RECORD — update the answer using this valid window

> **The key insight:** Each element is added ONCE (right pointer) and removed AT MOST ONCE (left pointer). Both pointers only move forward. That's why the total work is O(n) — even though there's an inner while-loop, it can never run more than n times total across the entire outer loop.

7. Variable-Size — Problem 1: Longest Substring Without Repeating Characters

Given a string, find the length of the longest substring with all UNIQUE characters.

Example

```
     s = "abcabcbb"

     Expand right, keep unique chars in the window:
        window "a"           → length 1
        window "ab"          → length 2
        window "abc"         → length 3    ← longest so far
        try adding 'a'       → duplicate! shrink until "bc"
        window "bca"         → length 3
        try adding 'b'       → duplicate! shrink until "ca"
        ...

     Answer: 3  ("abc")
```

Code

```
public static int lengthOfLongestSubstring(String s) {
    Set<Character> window = new HashSet<>();
    int left = 0, best = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);

        // Shrink while duplicate
        while (window.contains(c)) {
            window.remove(s.charAt(left));
            left++;
        }

        window.add(c);
        best = Math.max(best, right - left + 1);
    }

    return best;
}

// Time: O(n)
// Space: O(min(n, alphabet))
```

8. Variable-Size — Problem 2: Smallest Subarray with Sum ≥ Target

Given a positive-integer array and a target, find the SHORTEST contiguous subarray whose sum is ≥ target. Return 0 if no such subarray exists.

Example

```
arr = [2, 3, 1, 2, 4, 3],  target = 7

Expand until sum >= 7:
   window [2,3,1,2] sum=8  ✓  length=4
   Shrink from left → [3,1,2] sum=6 < 7 stop shrinking

Continue expanding:
   [3,1,2,4] sum=10  ✓ length=4
   Shrink → [1,2,4] sum=7 ✓ length=3
   Shrink → [2,4] sum=6 < 7 stop

Continue:
   [2,4,3] sum=9  ✓  length=3
   Shrink → [4,3] sum=7 ✓ length=2    ← smallest!

Answer: 2
```

Code

```
public static int minSubArrayLen(int target, int[] arr) {
    int left = 0, sum = 0, best = Integer.MAX_VALUE;

    for (int right = 0; right < arr.length; right++) {
        sum += arr[right];

        while (sum >= target) {
            best = Math.min(best, right - left + 1);
            sum -= arr[left];
            left++;
        }
    }

    return best == Integer.MAX_VALUE ? 0 : best;
}
```

Positive numbers only: This shrink-when-large strategy REQUIRES non-negative numbers. If the array has negatives, adding more elements might DECREASE the sum — breaking the monotonic behavior sliding window depends on. For those problems, use prefix sums + HashMap instead.

9. Variable-Size — Problem 3: Longest Substring with At Most K Distinct Characters

Find the longest substring containing AT MOST K distinct characters.

```
import java.util.*;

public static int longestKDistinct(String s, int k) {
    Map<Character, Integer> freq = new HashMap<>();
    int left = 0, best = 0;

    for (int right = 0; right < s.length(); right++) {
        char c = s.charAt(right);
        freq.put(c, freq.getOrDefault(c, 0) + 1);

        // Shrink while we have too many distinct characters
        while (freq.size() > k) {
            char l = s.charAt(left);
            freq.put(l, freq.get(l) - 1);
            if (freq.get(l) == 0) freq.remove(l);
            left++;
        }

        best = Math.max(best, right - left + 1);
    }

    return best;
}

// Example:
//   s = "eceba", k = 2
//   longest = "ece" → length 3
```

> **The freq map is the window state:** For substring problems, a HashMap<Character, Integer> tracks how many of each character are currently in the window. When you shrink, decrement the count. When it hits zero, REMOVE the key so freq.size() correctly gives "distinct characters in window."

10. Variable-Size — Problem 4: Longest Repeating Character Replacement

Given a string and an integer K, you can change at most K characters. Find the length of the longest substring containing the same letter after replacements.

The Insight

A window is VALID if (window length - most frequent char count) ≤ K. That means the number of chars we'd have to replace to make everything the same is at most K.

```
public static int characterReplacement(String s, int k) {
    int[] count = new int[26];    // frequency of each letter in window
    int left = 0, best = 0, maxFreq = 0;

    for (int right = 0; right < s.length(); right++) {
        count[s.charAt(right) - 'A']++;
        maxFreq = Math.max(maxFreq, count[s.charAt(right) - 'A']);

        // If replacements needed > k, shrink
        if ((right - left + 1) - maxFreq > k) {
            count[s.charAt(left) - 'A']--;
            left++;
        }

        best = Math.max(best, right - left + 1);
    }

    return best;
}
```

11. Variable-Size — Problem 5: Permutation in String

Given two strings s1 and s2, return true if s2 contains any PERMUTATION of s1. Uses a fixed-size window (size = s1.length()) with frequency comparison.

```
public static boolean checkInclusion(String s1, String s2) {
    if (s1.length() > s2.length()) return false;

    int[] s1Count = new int[26];
    int[] windowCount = new int[26];

    // Initialize with first window
    for (int i = 0; i < s1.length(); i++) {
        s1Count[s1.charAt(i) - 'a']++;
        windowCount[s2.charAt(i) - 'a']++;
    }

    if (Arrays.equals(s1Count, windowCount)) return true;

    // Slide the window
    for (int right = s1.length(); right < s2.length(); right++) {
        windowCount[s2.charAt(right) - 'a']++;
        windowCount[s2.charAt(right - s1.length()) - 'a']--;

        if (Arrays.equals(s1Count, windowCount)) return true;
    }

    return false;
```

```
}

// Time: O(n) with 26-element array comparisons
```

12. How to Spot a Sliding Window Problem

Signals that scream "use sliding window":

- Problem mentions CONTIGUOUS subarray or SUBSTRING
- You're asked for MAX / MIN / LONGEST / SHORTEST size satisfying a condition
- You're asked to COUNT subarrays satisfying a condition
- Brute force feels like $O(n^2)$ with nested loops examining every subarray
- The condition can be updated INCREMENTALLY as you add/remove one element
- The condition is MONOTONIC — becomes worse as you add, better as you remove

> **When it does NOT work:** Sliding window fails when adding an element could either help OR hurt the objective unpredictably. For example, subarray sum with NEGATIVE numbers doesn't play nicely — adding more elements might shrink the sum. In those cases, use prefix sums + HashMap instead.

13. Sliding Window vs Two Pointers

Both use two indices. They're closely related — you can think of sliding window as a specialized two-pointer approach. The key differences:

Aspect	Sliding Window	Two Pointers
Pointer motion	Both move in the same direction (forward)	Can move toward each other or in either direction
Structure	Range [left, right] on contiguous data	Two independent indices
Typical use	Contiguous subarrays / substrings	Sorted arrays, palindromes, pair sums
Data must be...	Sequential (arrays, strings)	Often sorted

14. Complexity Analysis

Pattern	Time	Space
Fixed-size window	O(n)	O(1)
Variable-size window	O(n)	O(1) to O(k) depending on state

Pattern	Time	Space
Character frequency window	O(n)	O(1) for fixed alphabet, O(k) generally
Compared to brute force	O(n) vs $O(n^2)$ or O(n·k)	Massive win

The classic "aha" moment: even though variable-size window has a nested while-loop, its total time is O(n) because the LEFT pointer only ever moves forward.

15. Common Mistakes

Mistake 1: Recomputing the window every iteration

```
// BAD — O(n²)
for (int right = 0; right < n; right++) {
    int sum = 0;
    for (int i = left; i <= right; i++) sum += arr[i];   // recomputes!
    ...
}

// GOOD — O(n)
for (int right = 0; right < n; right++) {
    sum += arr[right];                    // incremental
    while (sum > target) sum -= arr[left++];
    ...
}
```

Mistake 2: Forgetting to update state when shrinking

When left moves forward, you MUST remove arr[left] from your window state (subtract from sum, decrement freq, etc.). Forgetting this leaves the state out of sync — bugs will follow.

Mistake 3: Using sliding window with negative numbers for sum problems

The "shrink while sum too big" trick fails if elements can be negative — adding more might DECREASE the sum. If negatives are allowed, use prefix sums + HashMap instead.

Mistake 4: Off-by-one when computing window length

```
int length = right - left;          // ✗ WRONG — this is 1 too small
int length = right - left + 1;      // ✓ CORRECT for [left, right]
inclusive
```

Mistake 5: Confusing fixed-size with variable-size

If the problem says "of size K," use fixed. If the size can change based on a condition, use variable. Some problems seem fixed but are actually variable (like "smallest subarray with sum ≥ target") — read carefully.

Mistake 6: Using while instead of if for fixed-size shrink

```
// BAD — for fixed-size, use if (not while)
while (right - left + 1 > k) left++;     // ✗ overkill
if (right - left + 1 > k) { ... left++; } // ✓ cleaner

// BUT for variable-size, use while
while (isInvalid) { ...; left++; }         // ✓ shrink until valid
```

Mistake 7: Not resetting state after moving on

If you use HashMap or Set for the window state, remember to remove entries (not just decrement counts to 0) so freq.size() stays accurate.

16. Best Practices

- Always ask FIRST: is this a fixed-size or variable-size problem?
- For fixed-size, use the delta trick: sum += arr[right] - arr[right-k]
- For variable-size, use the EXPAND/SHRINK/RECORD template — every time
- Choose window state carefully: sum for numeric, Set for uniqueness, HashMap for frequency
- For character problems with fixed alphabet, use int[26] instead of HashMap — faster
- Compute window length as right - left + 1 (inclusive on both ends)
- Update state BEFORE checking validity; check AFTER updating
- Sanity-check: your left pointer should only move FORWARD, never backward

17. Quick Reference

Concept	Meaning
Sliding Window	A range [left, right] moved across data
Fixed-size window	Window of a given size K
Variable-size window	Window that grows/shrinks based on a condition
Delta update	sum += arr[right] - arr[right - k]
Expand	Move right forward, add arr[right] to state
Shrink	Move left forward, remove arr[left] from state
Record	Update best answer based on current window
Window length	right - left + 1
Common state	sum (numeric), Set (unique chars), Map (frequency)

Concept	Meaning
Time complexity	O(n) — each element visited constant times

18. Practice Problems

Try each — they cover every common sliding window pattern.

Fixed-Size Windows

4. Maximum Sum Subarray of Size K.
5. Average of Subarrays of Size K.
6. First Negative Number in Every Window of Size K.
7. Count Occurrences of Anagrams (of pattern P in text T).
8. Maximum of All Subarrays of Size K (use Deque).

Variable-Size — Longest

9. Longest Substring Without Repeating Characters.
10. Longest Substring with At Most K Distinct Characters.
11. Longest Substring with At Most 2 Distinct Characters.
12. Longest Repeating Character Replacement.
13. Fruit into Baskets (variant of at-most-2 distinct).

Variable-Size — Shortest

14. Minimum Size Subarray Sum (sum ≥ target).
15. Minimum Window Substring (smallest window containing all chars of T).

Counting Windows

16. Count Subarrays with At Most K Distinct Elements.
17. Count Subarrays with Exactly K Distinct Elements (hint: atMost(K) - atMost(K-1)).
18. Number of Substrings Containing All Three Characters (a, b, c).

Advanced

19. Permutation in String.
20. Find All Anagrams in a String.
21. Substring with Concatenation of All Words.
22. Sliding Window Maximum (using monotonic deque).
23. Longest Subarray with Sum Divisible by K (uses prefix sums, not sliding — for comparison).

19. Final Takeaway

Sliding window is deceptively simple: a range that moves. But when you recognize the pattern, an entire category of "impossible" $O(n^2)$ problems collapses into elegant O(n) solutions. The three-step template — EXPAND, SHRINK, RECORD — handles nearly every variant.

Once you've solved 8-10 sliding window problems, they start looking obvious. The signal words ("longest," "shortest," "contiguous," "substring," "at most K") almost immediately map to a variant of the template. That pattern recognition is the entire skill.

> **Key Takeaway:** Two flavors: FIXED-SIZE (window slides K steps at a time) and VARIABLE-SIZE (expand while valid, shrink when invalid). Both are O(n). Template for variable-size: for each right → add to state → while invalid, remove arr[left++] → record best. Choose window state based on the problem: sum, Set, or HashMap. Master this pattern and you unlock a huge slice of interview problems.

20. What's Next?

With sliding window done, the remaining core patterns are:

- Two Pointers in depth (palindrome, container with most water, 3-sum)
- Prefix Sums — array sums with negative numbers
- Tries — prefix trees for autocomplete and word search
- Union-Find (DSU) — connectivity and grouping problems
- Bit Manipulation — subset generation with bitmasks, XOR tricks
- Monotonic Stack / Deque — next greater element, largest rectangle
- String algorithms — KMP, Rabin-Karp for fast pattern matching
- System Design — for the later interview rounds